

DSA Roadmap Practice Sheet

Phase 0 has concept notes. Phases 1–13 have curated interview-style problems (mostly LeetCode + GFG). Use **Hint** for a short approach cue – not a full solution – then open **Link** to solve it yourself.

[14 phases](#)[382 practice links](#)[16 Phase 0 notes](#)[398 subtopics](#)[Full coverage](#)

Jump to phase

- | | |
|---|--|
| 0. Phase 0 – Fundamentals | 7. Phase 7 – BST |
| 1. Phase 1 – Arrays & Strings | 8. Phase 8 – Heap / PQ |
| 2. Phase 2 – Binary Search | 9. Phase 9 – Graphs |
| 3. Phase 3 – Linked List | 10. Phase 10 – Greedy |
| 4. Phase 4 – Stack & Queue | 11. Phase 11 – Bit Manipulation |
| 5. Phase 5 – Recursion & BT | 12. Phase 12 – Trie |
| 6. Phase 6 – Trees | 13. Phase 13 – Dynamic Programming |

Fundamentals

Complexity, recursion basics, and the math you need before patterns.

1. Complexity Analysis

Time Complexity

Notes

Time complexity counts how operations grow with input size n . Focus on dominant term: $O(n)$, $O(n \log n)$, $O(n^2)$. Nested loops usually multiply; sequential blocks add. Always state the worst case unless asked otherwise.

Space Complexity

Notes

Space complexity is extra memory beyond the input. Recursion depth counts as $O(\text{depth})$ stack space. In-place algorithms aim for $O(1)$ extra (ignoring output). Clarify whether the answer array counts.

Big-O / Big-Theta / Big-Omega

Notes

Big-O = upper bound, Big-Omega = lower bound, Big-Theta = tight bound. Interviewers usually mean Big-O worst case. Prefer tight bounds when you know them ($\tilde{O}(n)$ vs $O(n^2)$).

Amortized Complexity

Notes

Amortized analysis spreads expensive rare ops over many cheap ones (e.g. dynamic array resize). Average cost per op can be $O(1)$ even if one op is $O(n)$.

2. Recursion Basics

Base Case

Notes

Every recursive function needs a base case that stops calling itself. Without it you get infinite recursion / stack overflow. Base cases are often empty input, $n=0/1$, or null node.

Recursive Case

Notes

The recursive case reduces the problem toward the base case ($n-1$, left/right half, smaller substring). Trust that the recursive call returns a correct answer for the smaller problem, then combine.

Call Stack

Notes

Each call pushes a frame (locals + return address). Deep recursion $\approx O(n)$ stack. Tail recursion is rarely optimized in JS – prefer iteration when depth is huge.

Parameterized Recursion

Notes

Parameterized recursion passes running state as arguments (index, sum so far). Useful when you build answers while going down the call tree.

Functional Recursion

Notes

Functional recursion returns a value and combines results on the way back (e.g. return left + right). Cleaner for tree/subproblem merges.

Multiple Recursive Calls

Notes

Multiple recursive calls (fibonacci, tree DFS) branch the call tree. Watch exponential blow-up – add memoization or convert to DP when overlapping subproblems appear.

3. Mathematical Basics

GCD / LCM

Notes

Euclidean GCD: $\text{gcd}(a,b) = \text{gcd}(b, a \% b)$. $\text{LCM}(a,b) = |a*b| / \text{gcd}(a,b)$ (watch overflow). Used in fractions, cycles, and tiling problems.

Prime Numbers

Notes

Trial division up to $\sqrt{n}$ checks primality. Factorization and prime checks show up in math-flavored interview warmups.

Sieve of Eratosthenes

Notes

Sieve of Eratosthenes marks multiples of each prime up to n in $O(n \log \log n)$. Precompute primes when many queries share the same n .

Modular Arithmetic

Notes

Modular arithmetic: $(a+b)\%m$, $(a*b)\%m$ with care for negatives in languages that differ. Fermat/modular inverse appear in combinatorics under prime moduli.

Fast Exponentiation

Notes

Binary exponentiation computes a^n in $O(\log n)$ by squaring. Same idea for modular pow and matrix exponentiation.

Basic Combinatorics

Notes

nCr , permutations, and Pascal identities underpin counting paths and DP transitions. Precompute factorials + inverse factorials for many queries mod p .

Arrays & Strings

Core array patterns – hashing, windows, prefixes, intervals, and matrices.

1. Array Basics

Traversal

Running Sum of 1d Array

[LEETCODE](#)[Link](#)

One pass left to right; keep a running total and write it into each index (or a new array).

In-place Modification

Move Zeroes

[LEETCODE](#)[Link](#)

Two pointers: write non-zeros forward, then fill the rest with zeros "no extra array."

Frequency Counting

Majority Element

[LEETCODE](#)[Link](#)

Boyer-Moore voting or a frequency map; majority appears more than $\lceil n/2 \rceil$ times.

Min / Max

Third Maximum Number

[LEETCODE](#)[Link](#)

Track top distinct values while scanning once; handle fewer than 3 distinct carefully.

Rotation

Rotate Array

[LEETCODE](#)[Link](#)

Reverse whole array, then reverse first k and the rest ($k \% = n$).

Rearrangement

Rearrange Array Elements by Sign

LEETCODE

[Link](#)

Place positives on even indices and negatives on odd (or two-pointer write).

Matrix / 2D Arrays

Matrix Diagonal Sum

LEETCODE

[Link](#)

Sum primary and secondary diagonals; if n is odd, subtract the center once.

2. Hashing

HashMap

Two Sum

LEETCODE

[Link](#)

Map value $\rightarrow$ index; for each num check if target $-$ num was already seen.

HashSet

Contains Duplicate

LEETCODE

[Link](#)

Insert into a HashSet; if insert fails, a duplicate exists.

Frequency Map

Valid Anagram

LEETCODE

[Link](#)

Count character frequencies; compare two maps (or one map with $+$ / $-$).

Index Lookup

First Unique Character in a String

LEETCODE

[Link](#)

Frequency pass, then scan for the first char with count 1.

Complement Lookup

Two Sum II - Input Array Is Sorted

LEETCODE

[Link](#)

Complement idea still works; with sorted input prefer two pointers from both ends.

Grouping

Group Anagrams

LEETCODE

[Link](#)

Key = sorted string (or count signature); group words under the same key.

Deduplication

Remove Duplicates from Sorted Array

LEETCODE

[Link](#)

Slow/fast pointers: write unique values forward in the sorted array.

3. Prefix Sum

Prefix Sum

Find Pivot Index

LEETCODE

[Link](#)

Total sum + left running sum: $\text{left} == \text{total} - \text{left} - \text{nums}[i]$.

Prefix Product

Product of Array Except Self

LEETCODE

[Link](#)

Prefix products left $\rightarrow$ right, then multiply by suffix products right $\rightarrow$ left (no division).

Range Sum

Range Sum Query - Immutable

LEETCODE

[Link](#)

Build `prefix[]`; range sum = $\text{prefix}[j+1] - \text{prefix}[i]$.

Prefix Sum + HashMap

Subarray Sum Equals K

LEETCODE

[Link](#)

Map `prefixSum $\rightarrow$ count`; for each prefix look up `prefix $\hat{=}$ k`.

Prefix XOR

XOR Queries of a Subarray

LEETCODE

[Link](#)

Prefix XOR: range XOR = $\text{pref}[r+1] \oplus \text{pref}[l]$.

2D Prefix Sum

Range Sum Query 2D - Immutable

LEETCODE

[Link](#)

2D prefix sums; answer with inclusion"exclusion of four corners.

4. Difference Array

Range Updates

Corporate Flight Bookings

LEETCODE

[Link](#)

Difference array: +seats at start, $-$ seats after end; then prefix sum.

Range Increment

Range Addition

LEETCODE

[Link](#)

Same difference-array trick for many range increments, then one prefix pass.

Difference Array + Prefix Sum

Car Pooling

LEETCODE

[Link](#)

Diff along timeline (pickup +, drop $-$), prefix to track passengers vs capacity.

5. Two Pointers

Opposite Direction

Two Sum II - Input Array Is Sorted

LEETCODE

[Link](#)

Left and right ends; move the side that makes the sum closer to target.

Same Direction

Remove Duplicates from Sorted Array II

LEETCODE

[Link](#)

Same-direction write pointer; allow at most two copies of each value.

Fast / Slow Pointer

Linked List Cycle

LEETCODE

[Link](#)

Floyd: slow +1, fast +2; meeting means a cycle (array version: happy number).

Sorted Array Pair Problems

Container With Most Water

LEETCODE

[Link](#)

Opposite pointers; move the shorter height inward to seek a larger area.

3Sum / 4Sum

3Sum

LEETCODE

[Link](#)

Sort, fix one index, two-pointer the rest; skip duplicates carefully.

Partitioning

Sort Colors

LEETCODE

[Link](#)

Dutch national flag: low/mid/high pointers partition 0s, 1s, 2s in one pass.

In-place Array Problems

Squares of a Sorted Array

LEETCODE

[Link](#)

Largest squares are at ends; fill result from right using two pointers.

6. Sliding Window

Fixed Size Window

Maximum Average Subarray I

LEETCODE

[Link](#)

Fixed window of size k: slide by adding right and removing left.

Variable Size Window

Longest Substring Without Repeating Characters

LEETCODE

[Link](#)

Expand right; shrink left when a duplicate appears (set/map of last index).

Expand / Shrink

Minimum Size Subarray Sum

LEETCODE

[Link](#)

Expand until sum $\geq$ target, then shrink to minimize length.

Frequency Map Window

Find All Anagrams in a String

LEETCODE

[Link](#)

Window size = p.length; maintain char counts and a 'matches' counter.

Character Window

Permutation in String

LEETCODE

[Link](#)

Same as anagram window: fixed length with frequency equality check.

Maximum / Minimum Window

Minimum Window Substring

LEETCODE

[Link](#)

Expand until all needed chars covered, then shrink for the minimum window.

Deque-based Window

Sliding Window Maximum

LEETCODE

[Link](#)

Monotonic decreasing deque of indices; front is max of current window.

7. Kadane / Subarray

Maximum Subarray

Maximum Subarray

[LEETCODE](#)[Link](#)

Kadane: at each index choose `nums[i]` alone or extend previous best sum.

Minimum Subarray

Maximum Absolute Sum of Any Subarray

[LEETCODE](#)[Link](#)

Track both max and min subarray sums (Kadane variants); answer is $\max(|\text{sum}|)$.

Maximum Circular Subarray

Maximum Sum Circular Subarray

[LEETCODE](#)[Link](#)

$\max(\text{normal Kadane}, \text{totalSum} - \text{minSubarraySum})$; handle all-negative edge case.

Maximum Product Subarray

Maximum Product Subarray

[LEETCODE](#)[Link](#)

Track max and min product ending here (negatives flip them).

Subarray Sum Problems

Continuous Subarray Sum

LEETCODE

[Link](#)

Prefix sums mod k ; same remainder seen before $\Rightarrow$ subarray multiple of k (length ≥ 2).

8. Sorting

Bubble Sort

Sort an Array

LEETCODE

[Link](#)

Implement any $O(n \log n)$ sort; bubble is for learning swaps, not interviews.

Selection Sort

Sort Colors

LEETCODE

[Link](#)

Selection-style thinking: repeatedly place the next correct value " here use 3-way partition.

Insertion Sort

Insertion Sort List

LEETCODE

[Link](#)

Build a sorted list by inserting each node into the correct position.

Merge Sort

Sort List

LEETCODE

[Link](#)

Merge sort on linked list: split mid, sort halves, merge.

Quick Sort

Kth Largest Element in an Array

LEETCODE

[Link](#)

Quickselect (partition like quicksort) for average $O(n)$; or heap.

Counting Sort

Sort Colors

LEETCODE

[Link](#)

Counting sort works when values are few (0/1/2) â€” count then overwrite.

Custom Comparator

Largest Number

LEETCODE

[Link](#)

Custom comparator: order A before B if $AB > BA$ as strings.

Sort + Pattern

Sort + Two Pointers

3Sum Closest

LEETCODE

[Link](#)

Sort first, then fix one index and two-pointer for closest sum.

Sort + Greedy

Assign Cookies

LEETCODE

[Link](#)

Sort greed and sizes; assign smallest cookie that satisfies each child.

Sort + Intervals

Merge Intervals

LEETCODE

[Link](#)

Sort by start; merge when current overlaps the last kept interval.

9. Intervals

Merge Intervals

Merge Intervals

LEETCODE

[Link](#)

Sort by start time; extend end while overlapping.

Insert Interval

Insert Interval

LEETCODE

[Link](#)

Add all before, merge overlaps with newInterval, then append the rest.

Overlapping Intervals

Meeting Rooms II

LEETCODE

[Link](#)

Sort starts/ends or use min-heap of end times to count concurrent meetings.

Non-overlapping Intervals

Non-overlapping Intervals

LEETCODE

[Link](#)

Sort by end; greedily keep intervals that don't overlap the last kept.

Meeting Rooms

Meeting Rooms

LEETCODE

[Link](#)

Sort by start; check if any meeting starts before the previous ends.

Interval Scheduling

Minimum Number of Arrows to Burst Balloons

LEETCODE

[Link](#)

Sort by end; shoot at end of current balloon; skip all covered by that point.

Sweep Line

The Skyline Problem

LEETCODE

[Link](#)

Sweep line on building edges with a multiset/heap of active heights.

10. Matrices

Matrix Traversal

Island Perimeter

LEETCODE

[Link](#)

Traverse grid; for each land cell add edges not shared with another land.

Spiral Matrix

Spiral Matrix

LEETCODE

[Link](#)

Simulate boundaries top/bottom/left/right and shrink after each side.

Rotate Matrix

Rotate Image

LEETCODE

[Link](#)

Transpose, then reverse each row (for 90° clockwise).

Transpose

Transpose Matrix

LEETCODE

[Link](#)

`result[j][i] = matrix[i][j]`; allocate $m \times n$ carefully when not square.

Search Matrix

Search a 2D Matrix

LEETCODE

[Link](#)

Treat as a flattened sorted array and binary search, or start top-right.

Set Matrix Zeroes

Set Matrix Zeroes

LEETCODE

[Link](#)

Mark rows/cols using first row & first column as flags ($O(1)$ extra space).

Grid-based Problems

Number of Islands

LEETCODE

[Link](#)

DFS/BFS flood-fill each unvisited land cell; count components.

Binary Search

Standard search, rotated arrays, and binary search on the answer space.

1. Basic Binary Search

Standard Binary Search

Binary Search

[LEETCODE](#)[Link](#)

Classic $mid = lo + (hi - lo) / 2$; move lo/hi based on comparison with target.

First Occurrence

Find First and Last Position of Element in Sorted Array

[LEETCODE](#)[Link](#)

Two binary searches: `lower_bound` for first, then `upper_bound - 1` for last.

Last Occurrence

Find First and Last Position of Element in Sorted Array

[LEETCODE](#)[Link](#)

For last occurrence, bias the search to the right when `nums[mid] == target`.

Lower Bound

Search Insert Position

[LEETCODE](#)[Link](#)

Lower bound: first index with value $\geq$ target.

Upper Bound

Count Negative Numbers in a Sorted Matrix

[LEETCODE](#)[Link](#)

Per row, find first negative (upper/lower bound style) or staircase from top-right.

Search Insert Position

Search Insert Position

LEETCODE

[Link](#)

Binary search until $lo > hi$; lo is the insert index.

2. Modified Binary Search

Rotated Sorted Array

Search in Rotated Sorted Array

LEETCODE

[Link](#)

Decide which half is sorted; check if target lies in that half.

Find Minimum in Rotated Array

Find Minimum in Rotated Sorted Array

LEETCODE

[Link](#)

Compare mid with right; the unsorted side contains the minimum.

Peak Element

Find Peak Element

LEETCODE

[Link](#)

Move toward the larger neighbor "a peak must exist that way."

Bitonic Array

Peak Index in a Mountain Array

LEETCODE

[Link](#)

Binary search the mountain peak where $arr[mid] > arr[mid+1]$ flips.

Nearly Sorted Array

Search in Rotated Sorted Array II

LEETCODE

[Link](#)

Duplicates blur halves; shrink lo/hi when `nums[lo] == nums[mid] == nums[hi]`.

3. Binary Search on Answer

Minimum Possible Answer

Koko Eating Bananas

LEETCODE

[Link](#)

Binary search speed; feasible(mid) checks if hours $\leq h$.

Maximum Possible Answer

Capacity To Ship Packages Within D Days

LEETCODE

[Link](#)

Search capacity; minimize the smallest capacity that finishes in $\leq d$ days.

Feasibility Function

Split Array Largest Sum

LEETCODE

[Link](#)

Binary search the max-sum limit; greedy count how many subarrays needed.

Capacity Problems

Capacity To Ship Packages Within D Days

LEETCODE

[Link](#)

Low = max package, high = sum; check days needed for mid capacity.

Allocation Problems

Split Array Largest Sum

LEETCODE

[Link](#)

Same pattern as book allocation: minimize largest sum across m splits.

Koko / Eating Problems

Koko Eating Bananas

LEETCODE

[Link](#)

Minimize eating speed with a time-feasibility check.

Aggressive Placement Problems

Magnetic Force Between Two Balls

LEETCODE

[Link](#)

Sort positions; binary search minimum distance; greedily place balls.

4. Binary Search + Other Patterns

Binary Search + Greedy

Minimum Number of Days to Make m Bouquets

LEETCODE

[Link](#)

Binary search day; greedily count bouquets formable that day.

Binary Search + Prefix Sum

Path With Minimum Effort

LEETCODE

[Link](#)

Binary search effort limit + BFS/DFS reachability (or Dijkstra).

Binary Search + Sorting

Successful Pairs of Spells and Potions

LEETCODE

[Link](#)

Sort potions; for each spell binary search the first potion that works.

Binary Search + Two Pointers

Heaters

LEETCODE

[Link](#)

Sort both; for each house binary search nearest heater (or two-pointer).

Linked List

Pointers, reversal, merging, and classic linked-list designs.

1. Linked List Basics

Traversal

Middle of the Linked List

LEETCODE

[Link](#)

Traverse with fast/slow, or count length then walk to mid.

Insert

Design Linked List

LEETCODE

[Link](#)

Dummy head helps insert at head/index without special cases.

Delete

Delete Node in a Linked List

LEETCODE

[Link](#)

Copy next's value into current, then bypass next (no head access).

Dummy Node

Remove Linked List Elements

LEETCODE

[Link](#)

Dummy node before head simplifies deleting the real head.

2. Fast / Slow Pointer

Find Middle

Middle of the Linked List

LEETCODE

[Link](#)

Slow moves 1, fast moves 2; slow lands on middle.

Cycle Detection

Linked List Cycle

LEETCODE

[Link](#)

Floyd cycle detection with slow/fast pointers.

Find Cycle Start

Linked List Cycle II

LEETCODE

[Link](#)

After meet, reset one pointer to head; advance both by 1 to find entrance.

Palindrome

Palindrome Linked List

LEETCODE

[Link](#)

Find mid, reverse second half, compare both halves.

3. Reversal

Reverse Entire List

Reverse Linked List

LEETCODE

[Link](#)

Iterative: prev/curr/next rewiring; or recursive reverse of rest.

Reverse Between Positions

Reverse Linked List II

LEETCODE

[Link](#)

Locate segment, reverse left..right in place, reconnect ends.

Reverse in K Groups

Reverse Nodes in k-Group

LEETCODE

[Link](#)

Only reverse a full group of k; recurse/iterate on the remainder.

Reverse Alternate Groups

Swap Nodes in Pairs

LEETCODE

[Link](#)

Special case of k=2: swap pairs and advance.

4. Merging

Merge Two Sorted Lists

Merge Two Sorted Lists

LEETCODE

[Link](#)

Dummy tail; always attach the smaller head of the two lists.

Merge K Sorted Lists

Merge k Sorted Lists

LEETCODE

[Link](#)

Min-heap of heads, or pairwise merge (divide & conquer).

Sort Linked List

Sort List

LEETCODE

[Link](#)

Merge sort: mid split via slow/fast, merge sorted halves.

5. Linked List Design

Intersection

Intersection of Two Linked Lists

LEETCODE

[Link](#)

Two pointers switch heads after end â€” they meet at intersection (or null).

Remove Nth Node

Remove Nth Node From End of List

LEETCODE

[Link](#)

Fast ahead by n; when fast ends, slow is before the node to delete.

Reorder List

Reorder List

LEETCODE

[Link](#)

Mid â†’ reverse second half â†’ weave the two halves.

Clone List with Random Pointer

Copy List with Random Pointer

LEETCODE

[Link](#)

Map oldâ†’new nodes, or interleave clones then assign randoms.

LRU Cache

LRU Cache

LEETCODE

[Link](#)

HashMap + doubly linked list for O(1) get/put and eviction.

Doubly Linked List

Design Browser History

LEETCODE

Link

Doubly linked nodes (or two stacks) for back/forward navigation.

Stack & Queue

Stacks, monotonic stacks, queues, and deque window tricks.

1. Stack

Basic Stack

Implement Stack using Queues

LEETCODE

[Link](#)

Keep newest at front via rotate-on-push, or rotate-on-pop.

Valid Parentheses

Valid Parentheses

LEETCODE

[Link](#)

Push opens; on close, stack top must be the matching open.

Min Stack

Min Stack

LEETCODE

[Link](#)

Store pairs (val, minSoFar) or maintain a parallel min stack.

Expression Evaluation

Basic Calculator II

LEETCODE

[Link](#)

Stack numbers; apply $*$ / immediately, $+$ $-$ push signed values, sum at end.

Infix / Prefix / Postfix

Evaluate Reverse Polish Notation

LEETCODE

[Link](#)

Stack operands; on operator pop two, push result.

Stack-based Design

Max Stack

LEETCODE

[Link](#)

Track max alongside values (two stacks or nodes with max links).

2. Monotonic Stack

Next Greater Element

Next Greater Element I

LEETCODE

[Link](#)

Monotonic decreasing stack over `nums2`; map each value to its next greater.

Next Smaller Element

Next Greater Element II

LEETCODE

[Link](#)

Circular array: iterate $2n$ with $i \% n$ and a decreasing stack.

Previous Greater Element

Daily Temperatures

LEETCODE

[Link](#)

Stack of indices waiting for a warmer day (previous greater pattern inverted).

Previous Smaller Element

Sum of Subarray Minimums

LEETCODE

[Link](#)

For each element find previous/next smaller to count subarrays where it is min.

Daily Temperatures

Daily Temperatures

LEETCODE

[Link](#)

Monotonic stack of unresolved colder days; `answer[i] = j - i` when warmer found.

Stock Span

Online Stock Span

LEETCODE

[Link](#)

Stack of (price, span); pop while price $\geq$ current and accumulate spans.

Largest Rectangle in Histogram

Largest Rectangle in Histogram

LEETCODE

[Link](#)

Increasing stack of indices; when smaller bar appears, pop and compute width.

Sum of Subarray Minimums

Sum of Subarray Minimums

LEETCODE

[Link](#)

Contribution technique with previous/next strictly smaller bounds.

Trapping Rain Water

Trapping Rain Water

LEETCODE

[Link](#)

Two pointers with leftMax/rightMax, or stack of bars for trapped width $\times$ height.

3. Queue

Basic Queue

Implement Queue using Stacks

LEETCODE

[Link](#)

Two stacks: in for push, out for pop/peek (transfer when out empty).

Circular Queue

Design Circular Queue

LEETCODE

[Link](#)

Array + front/rear indices with modulo wrapping; track size.

Queue using Stack

Implement Queue using Stacks

LEETCODE

[Link](#)

Amortized $O(1)$: flush input stack into output only when needed.

Stack using Queue

Implement Stack using Queues

LEETCODE

[Link](#)

On push, enqueue then rotate older elements behind the new one.

4. Deque / Monotonic Queue

Sliding Window Maximum

Sliding Window Maximum

LEETCODE

[Link](#)

Monotonic deque stores candidates in decreasing order.

Sliding Window Minimum

Shortest Subarray with Sum at Least K

LEETCODE

[Link](#)

Prefix sums + increasing deque to find shortest valid window.

First Negative in Window

First Negative Integer in Every Window of Size K

GFG

[Link](#)

Deque stores indices of negatives; drop those outside the window.

Monotonic Queue Optimization

Jump Game VI

LEETCODE

[Link](#)

DP + monotonic deque of best scores within last k indices.

Recursion & Backtracking

Pick/skip trees, subsets, permutations, and constraint search.

1. Recursion Patterns

Pick / Not Pick

Subsets

LEETCODE

[Link](#)

At each index: pick or not pick, then recurse; collect when done.

Take / Skip

Combination Sum

LEETCODE

[Link](#)

Take (reuse same index) or skip to next; prune when remaining < 0 .

Multiple Choices

Letter Combinations of a Phone Number

LEETCODE

[Link](#)

For each digit, branch over its letters; build string along the path.

Recursive Tree

Generate Parentheses

LEETCODE

[Link](#)

Recursive tree of '(' / ')'; only add if open/close counts stay valid.

State Tracking

Word Search

LEETCODE

[Link](#)

DFS with visited state on the board; backtrack after exploring.

2. Subsets / Subsequences

Subsets

Subsets

[LEETCODE](#)[Link](#)

Backtracking or bit mask over all 2^n subsets.

Subsets II

Subsets II

[LEETCODE](#)[Link](#)

Sort first; skip duplicate values at the same depth.

Subsequences

Distinct Subsequences

[LEETCODE](#)[Link](#)

Count ways s forms t " DP/recursion with pick matching chars.

Subsequence with Target

Combination Sum III

[LEETCODE](#)[Link](#)

Pick distinct "9; stop when count/k or sum constraints break.

Combination Sum

Combination Sum

LEETCODE

[Link](#)

Unlimited reuse; recurse with same start index after picking.

3. Permutations

Permutations

Permutations

[LEETCODE](#)[Link](#)

Swap-based or used[] boolean; build all orderings.

Permutations II

Permutations II

[LEETCODE](#)[Link](#)

Sort; skip a duplicate if the previous identical wasn't used.

Swap-based Permutation

Permutations

[LEETCODE](#)[Link](#)

For i in [start..n): swap start with i, recurse start+1, swap back.

Next Permutation

Next Permutation

[LEETCODE](#)[Link](#)

Find pivot, swap with next greater to right, reverse suffix.

4. Backtracking

Combination Problems

Combination Sum II

LEETCODE

[Link](#)

Each number once; sort and skip duplicates at the same level.

N-Queens

N-Queens

LEETCODE

[Link](#)

Place row by row; track cols and diagonals as forbidden sets.

Sudoku

Sudoku Solver

LEETCODE

[Link](#)

Try digits 1-9 in empty cells; validate row/col/box; backtrack.

Word Search

Word Search

LEETCODE

[Link](#)

Start DFS from each cell matching word[0]; mark visited then undo.

Maze Problems

Rat in a Maze Problem - I

GFG

[Link](#)

DFS four directions; mark path; collect strings of moves when goal reached.

Constraint Satisfaction

Beautiful Arrangement

LEETCODE

[Link](#)

Backtrack permutations with divisibility constraints pruned early.

5. Divide & Conquer

Merge Sort

Sort an Array

LEETCODE

[Link](#)

Merge sort: split, sort halves, merge in $O(n)$.

Quick Sort

Kth Largest Element in an Array

LEETCODE

[Link](#)

Quickselect partition until pivot index is $n^{\wedge}k$.

Count Inversions

Reverse Pairs

LEETCODE

[Link](#)

During merge sort, count cross-half pairs before merging.

QuickSelect

Kth Largest Element in an Array

LEETCODE

[Link](#)

QuickSelect: average $O(n)$ selection via partitioning.

Recursive Binary Search

Binary Search

LEETCODE

[Link](#)

Recursive binary search on $[lo, hi]$ mid split.

Trees

Traversals, properties, paths, views, and construction.

1. Tree Traversal

Preorder

Binary Tree Preorder Traversal

[LEETCODE](#)[Link](#)

Root → left → right (recursive or stack).

Inorder

Binary Tree Inorder Traversal

[LEETCODE](#)[Link](#)

Left → root → right; BST inorder is sorted.

Postorder

Binary Tree Postorder Traversal

[LEETCODE](#)[Link](#)

Left → right → root; two-stack or carefully ordered single stack.

Iterative Traversal

Binary Tree Inorder Traversal

[LEETCODE](#)[Link](#)

Explicit stack: go left, pop/visit, go right.

Morris Traversal

Binary Tree Inorder Traversal

[LEETCODE](#)[Link](#)

Morris: thread predecessor → current, visit, then restore links (O(1) extra).

Level Order / BFS

Binary Tree Level Order Traversal

LEETCODE

[Link](#)

BFS queue; process level size each round.

2. Tree Properties

Maximum Depth

Maximum Depth of Binary Tree

LEETCODE

[Link](#)

$1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right})).$

Minimum Depth

Minimum Depth of Binary Tree

LEETCODE

[Link](#)

BFS to first leaf is easiest; recursion must handle one-child nodes.

Balanced Tree

Balanced Binary Tree

LEETCODE

[Link](#)

Return height or -1 if unbalanced; check $|\text{leftHeight} - \text{rightHeight}| \leq 1$ bottom-up.

Diameter

Diameter of Binary Tree

LEETCODE

[Link](#)

At each node, $\text{path} = \text{leftHeight} + \text{rightHeight}$; track global max.

Maximum Path Sum

Binary Tree Maximum Path Sum

LEETCODE

[Link](#)

Return max gain upward; update global with left+node+right as bend path.

3. Tree Path Patterns

Root → Leaf

Path Sum

[LEETCODE](#)[Link](#)

DFS subtract node.val; true if leaf and remaining == 0.

Root → Node

Binary Tree Paths

[LEETCODE](#)[Link](#)

DFS build path string/list from root to each leaf.

Path Sum

Path Sum II

[LEETCODE](#)[Link](#)

Backtrack collecting all root→leaf paths that sum to target.

All Paths

Binary Tree Paths

[LEETCODE](#)[Link](#)

Enumerate every root-to-leaf path.

Maximum / Minimum Path

Sum Root to Leaf Numbers

LEETCODE

[Link](#)

Pass running number $10 * \text{prev} + \text{val}$; add at leaves.

4. Tree Views

Left View

Binary Tree Right Side View

LEETCODE

[Link](#)

Left view = first node per level in BFS (mirror of right-side view).

Right View

Binary Tree Right Side View

LEETCODE

[Link](#)

BFS: last node of each level is the right side.

Top View

Top View of Binary Tree

GFG

[Link](#)

BFS with horizontal distance; first node seen per HD is top view.

Bottom View

Bottom View of Binary Tree

GFG

[Link](#)

BFS with HD; last node seen per HD is bottom view.

Boundary Traversal

Boundary Traversal of Binary Tree

GFG

[Link](#)

Left boundary + leaves + reverse right boundary (avoid doubles).

5. Tree Relationships

Lowest Common Ancestor

Lowest Common Ancestor of a Binary Tree

LEETCODE

[Link](#)

If node is p/q return it; if both sides return non-null, node is LCA.

Distance Between Nodes

Minimum Distance Between BST Nodes

LEETCODE

[Link](#)

For general tree: path via LCA; for BST, inorder adjacent diffs.

Ancestors

Lowest Common Ancestor of a Binary Tree

LEETCODE

[Link](#)

DFS returning whether target found builds ancestor awareness.

Subtree Problems

Subtree of Another Tree

LEETCODE

[Link](#)

For each node, check identical-tree match against subRoot.

6. Tree Construction / Serialization

Serialize / Deserialize

Serialize and Deserialize Binary Tree

LEETCODE

[Link](#)

Preorder/BFS with null markers; parse tokens back into tree.

Construct from Preorder + Inorder

Construct Binary Tree from Preorder and Inorder Traversal

LEETCODE

[Link](#)

Preorder[0] is root; split inorder by root; recurse on ranges.

Construct from Inorder + Postorder

Construct Binary Tree from Inorder and Postorder Traversal

LEETCODE

[Link](#)

Postorder last is root; split inorder; build right then left carefully.

Flatten Binary Tree

Flatten Binary Tree to Linked List

LEETCODE

[Link](#)

Preorder weave: attach left chain then right, or reverse postorder linking.

Binary Search Tree

BST ops, order statistics, relationships, and iterators.

1. BST Basics

Search

Search in a Binary Search Tree

[LEETCODE](#)[Link](#)

Go left if target < node, else right " O(h).

Insert

Insert into a Binary Search Tree

[LEETCODE](#)[Link](#)

Walk to null child position and attach the new node.

Delete

Delete Node in a BST

[LEETCODE](#)[Link](#)

0/1 child: rewire; 2 children: replace with inorder successor then delete it.

Validate BST

Validate Binary Search Tree

[LEETCODE](#)[Link](#)

Pass valid (min,max) bounds down, or check inorder is strictly increasing.

2. BST Order

Kth Smallest

Kth Smallest Element in a BST

LEETCODE

[Link](#)

Inorder traversal; return the k-th visited node.

Kth Largest

Kth Smallest Element in a BST

LEETCODE

[Link](#)

Kth largest = reverse inorder (right+root+left) for k visits.

Inorder Successor

Inorder Successor in BST

LEETCODE

[Link](#)

If right child exists, leftmost of right; else walk ancestors upward.

Inorder Predecessor

Inorder Successor in BST II

LEETCODE

[Link](#)

Predecessor is mirror of successor: left subtree max or parent climb.

3. BST Relationships

LCA in BST

Lowest Common Ancestor of a Binary Search Tree

LEETCODE

[Link](#)

Walk down until p and q split left/right of current node.

Two Sum in BST

Two Sum IV - Input is a BST

LEETCODE

[Link](#)

HashSet while DFS, or two iterators from smallest/largest.

Pair with Given Sum

Two Sum IV - Input is a BST

LEETCODE

[Link](#)

Same as two-sum in BST: look for complement $k - val$.

4. BST Design

BST Iterator

Binary Search Tree Iterator

LEETCODE

[Link](#)

Stack of left spine; next pops and pushes left spine of right child.

Balanced BST Concepts

Balance a Binary Search Tree

LEETCODE

[Link](#)

Inorder to sorted array, then build balanced BST from mid.

Convert Sorted Array / List to BST

Convert Sorted Array to Binary Search Tree

LEETCODE

[Link](#)

Always pick mid as root for height balance.

Heap / Priority Queue

Heaps, Top-K, two-heap median, and k-way merge.

1. Heap Basics

Min Heap

Kth Largest Element in a Stream

LEETCODE

[Link](#)

Min-heap of size k stores the k largest; root is k th largest.

Max Heap

Last Stone Weight

LEETCODE

[Link](#)

Max-heap: smash two heaviest until one (or zero) remains.

Insert

Kth Largest Element in a Stream

LEETCODE

[Link](#)

Each add: push then pop if size $> k$.

Delete

Last Stone Weight

LEETCODE

[Link](#)

Repeated extract-max (delete) from heap until done.

Heapify

Sort an Array

LEETCODE

[Link](#)

Heap sort: build-heap then repeatedly extract max.

Heap Sort

Sort an Array

LEETCODE

[Link](#)

Implement heap sort for practice ($O(n \log n)$ in-place-ish).

2. Top K Pattern

Kth Largest

Kth Largest Element in an Array

LEETCODE

[Link](#)

Min-heap of size k, or Quickselect.

Kth Smallest

Kth Smallest Element in a Sorted Matrix

LEETCODE

[Link](#)

Min-heap merge of rows, or binary search on value.

Top K Frequent

Top K Frequent Elements

LEETCODE

[Link](#)

Count frequencies, then heap or bucket by frequency.

K Closest

K Closest Points to Origin

LEETCODE

[Link](#)

Max-heap of size k by distance, or quickselect on distance.

K Largest / Smallest Pairs

Find K Pairs with Smallest Sums

LEETCODE

[Link](#)

Min-heap of pairs (i,j) ; expand neighbors carefully to avoid dupes.

3. Two Heaps

Median from Data Stream

Find Median from Data Stream

LEETCODE

[Link](#)

Max-heap (low half) + min-heap (high half); balance sizes.

Running Median

Find Median from Data Stream

LEETCODE

[Link](#)

Same two-heap structure for running median after each insert.

Partitioning Data

Sliding Window Median

LEETCODE

[Link](#)

Two heaps + lazy deletion to maintain window median.

4. K-Way Merge

Merge K Sorted Lists

Merge k Sorted Lists

LEETCODE

[Link](#)

Min-heap of current heads across k lists.

Kth Smallest in Sorted Matrix

Kth Smallest Element in a Sorted Matrix

LEETCODE

[Link](#)

Heap starts with first column; pop and push next in row.

Smallest Range from K Lists

Smallest Range Covering Elements from K Lists

LEETCODE

[Link](#)

Min-heap of current heads; track max; advance the min list.

5. Heap + Greedy

Scheduling

Task Scheduler

LEETCODE

[Link](#)

Max-heap by remaining count; cool-down with a queue of wait times.

Task Assignment

IPO

LEETCODE

[Link](#)

Sort by capital; unlock into max-heap of profits; pick best k times.

Connect Ropes

Minimum Cost of Ropes

GFG

[Link](#)

Always connect two smallest (min-heap) to minimize total cost.

Streaming Problems

Find Median from Data Stream

LEETCODE

[Link](#)

Streaming + heap is the classic pattern for online statistics.

Graphs

BFS/DFS, cycles, topo sort, shortest paths, MST, and DSU.

1. Graph Representation

Adjacency Matrix

Find the Town Judge

LEETCODE

[Link](#)

Degree arrays (or matrix): judge trusted by all, trusts nobody.

Adjacency List

Find if Path Exists in Graph

LEETCODE

[Link](#)

Build adjacency list, then BFS/DFS from source to target.

Edge List

Number of Provinces

LEETCODE

[Link](#)

Matrix or edge list → components via DFS/Union-Find.

Directed / Undirected

Course Schedule

LEETCODE

[Link](#)

Directed edges = prerequisites; detect cycle / topo sort.

2. BFS

Graph Traversal

Find if Path Exists in Graph

LEETCODE

[Link](#)

Standard BFS traversal with visited set.

Shortest Path " Unweighted

Word Ladder

LEETCODE

[Link](#)

BFS on word graph (1-letter neighbors) for shortest transformation.

Multi-source BFS

01 Matrix

LEETCODE

[Link](#)

Multi-source BFS from all 0s to compute distance for 1s.

Grid BFS

Shortest Path in Binary Matrix

LEETCODE

[Link](#)

Grid BFS (8 directions); first time reaching end is shortest.

Level-based Problems

Binary Tree Level Order Traversal

LEETCODE

[Link](#)

Level-size BFS pattern transfers directly to graphs.

3. DFS

Graph Traversal

Number of Provinces

LEETCODE

[Link](#)

DFS mark entire component from each unvisited node.

Connected Components

Number of Provinces

LEETCODE

[Link](#)

Count connected components with DFS/Union-Find.

Number of Provinces

Number of Provinces

LEETCODE

[Link](#)

IsConnected matrix: DFS/BFS or DSU.

Grid DFS

Max Area of Island

LEETCODE

[Link](#)

DFS returns size of island; track global max.

Flood Fill

Flood Fill

LEETCODE

[Link](#)

DFS/BFS recolor connected same-color cells.

4. Grid as Graph

Number of Islands

Number of Islands

LEETCODE

[Link](#)

Flood each '1' to '0'/visited; count starts.

Rotten Oranges

Rotting Oranges

LEETCODE

[Link](#)

Multi-source BFS from all rotten; minutes = levels.

Flood Fill

Flood Fill

LEETCODE

[Link](#)

Classic grid DFS recolor.

Surrounded Regions

Surrounded Regions

LEETCODE

[Link](#)

Mark O's connected to border as safe, then flip the rest.

Shortest Path in Grid

Shortest Path in Binary Matrix

LEETCODE

[Link](#)

BFS on grid cells with clear path.

5. Cycle Detection

Undirected → BFS

Detect Cycle in Undirected Graph

GFG

[Link](#)

BFS with parent tracking: visit neighbor already visited & parent → cycle.

Undirected → DFS

Detect Cycle in Undirected Graph

GFG

[Link](#)

DFS with parent; back-edge to visited non-parent → cycle.

Directed → DFS

Course Schedule

LEETCODE

[Link](#)

DFS colors: visiting again on recursion stack → directed cycle.

Directed → Kahn's Algorithm

Course Schedule

LEETCODE

[Link](#)

Kahn's BFS: if processed < n nodes, a cycle remains.

6. Bipartite Graph

BFS Coloring

Is Graph Bipartite?

LEETCODE

[Link](#)

BFS 2-coloring; conflict on same color edge $\hat{=}$ not bipartite.

DFS Coloring

Is Graph Bipartite?

LEETCODE

[Link](#)

DFS assign opposite color to neighbors; detect conflicts.

7. Topological Sort

DFS

Course Schedule II

LEETCODE

[Link](#)

DFS postorder push to stack; reverse for topo order (if no cycle).

Kahn's Algorithm

Course Schedule II

LEETCODE

[Link](#)

Indegree queue (Kahn); append nodes as indegree hits 0.

Course Schedule

Course Schedule

LEETCODE

[Link](#)

Can finish all courses iff the prerequisite graph is a DAG.

Dependency Problems

Alien Dictionary

LEETCODE

[Link](#)

Build char-order edges from sorted words; topo sort (handle cycles).

8. Shortest Path

BFS

Word Ladder

LEETCODE

[Link](#)

Unweighted shortest path = BFS levels.

Dijkstra

Network Delay Time

LEETCODE

[Link](#)

Dijkstra from k ; answer is max dist if all reachable.

Bellman-Ford

Cheapest Flights Within K Stops

LEETCODE

[Link](#)

Bellman-Ford style relax for $k+1$ iterations (or BFS+cost).

Floyd-Warshall

Find the City With the Smallest Number of Neighbors at a Threshold Distance

LEETCODE

[Link](#)

Floyd-Warshall all-pairs, then count cities within threshold.

0-1 BFS

Minimum Cost to Make at Least One Valid Path in a Grid

LEETCODE

[Link](#)

0-1 BFS (deque): cost 0 to front, cost 1 to back.

Shortest Path in DAG

Cheapest Flights Within K Stops

LEETCODE

[Link](#)

On a DAG, topo order + one-pass edge relaxation gives shortest paths.

9. Minimum Spanning Tree

Prim's Algorithm

Min Cost to Connect All Points

LEETCODE

[Link](#)

Prim: grow MST with a min-heap of edges from the tree.

Kruskal's Algorithm

Min Cost to Connect All Points

LEETCODE

[Link](#)

Kruskal: sort all edges, Union-Find to add non-cycle edges.

MST + DSU

Connecting Cities With Minimum Cost

LEETCODE

[Link](#)

Kruskal + DSU until one component; fail if not fully connected.

10. Disjoint Set Union

Find

Number of Provinces

LEETCODE

[Link](#)

DSU find with path compression; count unique roots.

Union

Redundant Connection

LEETCODE

[Link](#)

Union edges; the one whose ends already share a parent is redundant.

Path Compression

Number of Operations to Make Network Connected

LEETCODE

[Link](#)

DSU with path compression; need components $\hat{+}$ 1 spare cables.

Union by Rank

Redundant Connection

LEETCODE

[Link](#)

Union by rank/size keeps trees shallow “ same problem, clean DSU.

Union by Size

Accounts Merge

LEETCODE

[Link](#)

Union emails in an account; group by parent with size-aware union.

Dynamic Connectivity

Number of Islands II

LEETCODE

[Link](#)

Add land dynamically; union with neighbors and track component count.

11. Advanced Graph

Bridges

Critical Connections in a Network

LEETCODE

[Link](#)

Tarjan/DFS discovery & low-link; bridge if $\text{low}[v] > \text{disc}[u]$.

Articulation Points

Critical Connections in a Network

LEETCODE

[Link](#)

Articulation points use similar disc/low; different condition than bridges.

Strongly Connected Components

Critical Connections in a Network

LEETCODE

[Link](#)

Practice Tarjan/Kosaraju for SCCs on directed graphs after bridges.

Tarjan's Algorithm

Critical Connections in a Network

LEETCODE

[Link](#)

Single DFS with discovery times and low-link values (Tarjan-style).

Greedy

Interval and scheduling greed, plus heap-backed greedy patterns.

1. Greedy Basics

Local Optimal Choice

Jump Game

LEETCODE

[Link](#)

Track farthest reachable; greedily extend while $i \neq \text{farthest}$.

Sorting + Greedy

Assign Cookies

LEETCODE

[Link](#)

Sort both arrays; greedily satisfy smallest greed first.

Proof / Exchange Argument

Gas Station

LEETCODE

[Link](#)

If total gas $\geq$ total cost a solution exists; start after the worst prefix.

2. Interval Greedy

Activity Selection

Non-overlapping Intervals

LEETCODE

[Link](#)

Activity selection: sort by end, keep next compatible interval.

Meeting Rooms

Meeting Rooms II

LEETCODE

[Link](#)

Greedy rooms via min-heap of end times (or sweep line).

Non-overlapping Intervals

Non-overlapping Intervals

LEETCODE

[Link](#)

Remove fewest = keep most; sort by end and count conflicts.

Minimum Arrows

Minimum Number of Arrows to Burst Balloons

LEETCODE

[Link](#)

Sort by end; one arrow at current end covers overlapping balloons.

3. Scheduling Greedy

Job Sequencing

Maximum Profit in Job Scheduling

LEETCODE

[Link](#)

Sort by end; DP + binary search previous non-overlapping job.

Task Scheduling

Task Scheduler

LEETCODE

[Link](#)

Idle slots driven by most frequent task; formula or simulation.

Minimum Platforms

Minimum Platforms

GFG

[Link](#)

Sort arrivals & departures; sweep to track current platforms needed.

CPU Scheduling

Single-Threaded CPU

LEETCODE

[Link](#)

Sort by enqueue time; min-heap by processing time for available jobs.

4. Greedy + Heap

Task Scheduling

Task Scheduler

LEETCODE

[Link](#)

Max-heap of remaining frequencies with cooldown queue.

IPO / Capital Problems

IPO

LEETCODE

[Link](#)

Unlock affordable projects into a profit max-heap; pick k times.

Minimum Cost Problems

Minimum Cost to Connect Sticks

LEETCODE

[Link](#)

Always combine two smallest sticks (min-heap).

Resource Allocation

Process Tasks Using Servers

LEETCODE

[Link](#)

Two heaps: free servers by weight/index, busy by free time.

5. Other Greedy

Jump Game

Jump Game II

LEETCODE

[Link](#)

Greedy BFS-like ranges: end of current jump, farthest in window.

Gas Station

Gas Station

LEETCODE

[Link](#)

Reset start when tank goes negative; check total gas $\hat{=}$ cost.

Candy

Candy

LEETCODE

[Link](#)

Two passes: left-to-right then right-to-left for rating peaks.

Fractional Knapsack

Fractional Knapsack

GFG

[Link](#)

Sort by value/weight; take full items then a fraction of next.

Huffman Coding

Huffman Encoding

GFG

[Link](#)

Min-heap merge two smallest frequencies repeatedly; codes from tree.

Connect Ropes

Minimum Cost of Ropes

GFG

[Link](#)

Connect two smallest repeatedly; sum costs.

Bit Manipulation

Bit ops, XOR tricks, counting, and bitmask techniques.

1. Bit Basics

AND

Bitwise AND of Numbers Range

LEETCODE

[Link](#)

Right-shift left/right until equal; bits that differ become 0 in AND range.

OR

Minimum Flips to Make a OR b Equal to c

LEETCODE

[Link](#)

Per bit of c: decide flips on a/b to match OR result.

XOR

Single Number

LEETCODE

[Link](#)

XOR all numbers; duplicates cancel, unique remains.

NOT

Complement of Base 10 Integer

LEETCODE

[Link](#)

Flip bits up to the highest set bit (mask), not infinite ~.

Left Shift

Number of 1 Bits

LEETCODE

[Link](#)

Practice shifts: $n \& 1$, then $n \gg= 1$ (or Brian Kernighan).

Right Shift

Reverse Bits

LEETCODE

[Link](#)

Build result by shifting left and appending n's low bit repeatedly.

2. Bit Operations

Check Bit

Number of 1 Bits

LEETCODE

[Link](#)

Check bit i with $(n \gg i) \& 1$.

Set Bit

Single Number II

LEETCODE

[Link](#)

Bit counting / state machines to set bits of the unique number.

Clear Bit

Power of Two

LEETCODE

[Link](#)

$n \& (n \hat{=} 1)$ clears lowest set bit; power of two has one bit.

Toggle Bit

Complement of Base 10 Integer

LEETCODE

[Link](#)

Toggle relevant bits via XOR with a mask of 1s.

Remove Lowest Set Bit

Number of 1 Bits

LEETCODE

[Link](#)

Brian Kernighan: $n \&= (n \wedge 1)$ removes lowest set bit; count iterations.

3. XOR Patterns

Single Number

Single Number

LEETCODE

[Link](#)

XOR fold of the array.

Missing Number

Missing Number

LEETCODE

[Link](#)

XOR all indices and values (or with 0..n).

Two Unique Numbers

Single Number III

LEETCODE

[Link](#)

XOR all ' bitmask; split numbers by a differing bit into two groups.

XOR Range

XOR Queries of a Subarray

LEETCODE

[Link](#)

Prefix XOR for $O(1)$ range XOR.

4. Bit Counting

Count Set Bits

Counting Bits

[LEETCODE](#)[Link](#)

DP: $\text{bits}[i] = \text{bits}[i > 1] + (i \& 1)$.

Power of Two

Power of Two

[LEETCODE](#)[Link](#)

$n > 0 \ \&\& \ (n \ \& \ (n \hat{=} 1)) == 0$.

Power of Four

Power of Four

[LEETCODE](#)[Link](#)

Power of two AND set bit only in even positions (mask 0x5555...).

Hamming Distance

Hamming Distance

[LEETCODE](#)[Link](#)

Popcount of $x \text{ XOR } y$.

5. Bitmask

Subsets using Bits

Subsets

[LEETCODE](#)[Link](#)

For mask in $0..(1 < n) - 1$, include i if bit i set.

Submask Enumeration

Count Number of Maximum Bitwise-OR Subsets

[LEETCODE](#)[Link](#)

Enumerate subsets/submasks; track max OR and count.

State Compression

Shortest Path Visiting All Nodes

[LEETCODE](#)[Link](#)

BFS state = (node, visitedMask).

Bitmask DP

Partition to K Equal Sum Subsets

[LEETCODE](#)[Link](#)

Bitmask DP or backtracking with used mask over subset sums.

Trie

Prefix trees, dictionary search, and binary XOR tries.

1. Trie Basics

Insert

Implement Trie (Prefix Tree)

LEETCODE

[Link](#)

Children map/array per node; mark endOfWord on insert.

Search

Implement Trie (Prefix Tree)

LEETCODE

[Link](#)

Walk chars; search succeeds only if endOfWord at last node.

Starts With

Implement Trie (Prefix Tree)

LEETCODE

[Link](#)

startsWith = path exists (endOfWord optional).

Delete

Implement Trie (Prefix Tree)

LEETCODE

[Link](#)

Delete: clear end flag / prune nodes with no children (careful with shared prefixes).

2. Trie Applications

Prefix Search

Replace Words

LEETCODE

[Link](#)

Trie of dictionary roots; replace each word with shortest root prefix.

Word Dictionary

Design Add and Search Words Data Structure

LEETCODE

[Link](#)

Trie + DFS when '.' matches any child.

Longest Common Prefix

Longest Common Prefix

LEETCODE

[Link](#)

Vertical scan or Trie depth until a branch splits.

Word Search

Word Search II

LEETCODE

[Link](#)

Trie of words + board DFS; prune dead Trie branches.

Autocomplete

Search Suggestions System

LEETCODE

[Link](#)

Sort products or Trie; collect up to 3 suggestions per prefix.

3. Binary Trie

Maximum XOR

Maximum XOR of Two Numbers in an Array

LEETCODE

[Link](#)

Binary Trie of bits; greedily prefer opposite bit for max XOR.

Minimum XOR

Maximum XOR With an Element From Array

LEETCODE

[Link](#)

Offline queries + binary Trie inserted by increasing m limit.

XOR Pair Problems

Maximum XOR of Two Numbers in an Array

LEETCODE

[Link](#)

Insert then query each number against the Trie for best pair XOR.

Dynamic Programming

From 1D and knapsack to strings, trees, bitmasks, and advanced DP.

1. DP Fundamentals

Identify State

Climbing Stairs

[LEETCODE](#)[Link](#)

State = steps to reach i ; practice defining $dp[i]$ clearly first.

Identify Transition

House Robber

[LEETCODE](#)[Link](#)

Transition: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$.

Base Case

Fibonacci Number

[LEETCODE](#)[Link](#)

Base cases $dp[0]/dp[1]$ (or $F(0)/F(1)$) unlock the recurrence.

Memoization

Climbing Stairs

[LEETCODE](#)[Link](#)

Top-down recursion + memo map/array for overlapping subproblems.

Tabulation

House Robber

[LEETCODE](#)[Link](#)

Bottom-up loop filling dp from base cases.

Space Optimization

Climbing Stairs

LEETCODE

[Link](#)

Only last two states needed $\hat{=}$ $O(1)$ rolling variables.

2. 1D DP

Fibonacci

Fibonacci Number

LEETCODE

[Link](#)

Classic 1D DP / two variables.

Climbing Stairs

Climbing Stairs

LEETCODE

[Link](#)

$Ways(i) = ways(i-1) + ways(i-2)$.

Frog Jump

Frog Jump

LEETCODE

[Link](#)

DP/BFS on (index, lastJump); try jump-1, jump, jump+1.

House Robber

House Robber

LEETCODE

[Link](#)

Cannot take adjacent houses → choose take vs skip.

Maximum Non-Adjacent Sum

House Robber

LEETCODE

[Link](#)

Maximum non-adjacent sum is the house robber pattern.

Min Cost Problems

Min Cost Climbing Stairs

LEETCODE

[Link](#)

$$dp[i] = cost[i] + \min(dp[i-1], dp[i-2]).$$

3. Grid DP

Unique Paths

Unique Paths

[LEETCODE](#)[Link](#)

$$dp[i][j] = dp[i-1][j] + dp[i][j-1].$$

Unique Paths II

Unique Paths II

[LEETCODE](#)[Link](#)

Same as unique paths; obstacles force $dp = 0$.

Minimum Path Sum

Minimum Path Sum

[LEETCODE](#)[Link](#)

$$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1]).$$

Grid with Obstacles

Unique Paths II

[LEETCODE](#)[Link](#)

Treat obstacles as blocked cells in grid DP.

Dungeon Game

Dungeon Game

LEETCODE

[Link](#)

DP backward from princess: min HP needed entering each cell.

4. Knapsack DP

0/1 Knapsack

0-1 Knapsack Problem

GFG

[Link](#)

Classic 0/1: $dp[w] = \max(dp[w], dp[w - wt] + val)$ reverse iterate.

Unbounded Knapsack

Coin Change II

LEETCODE

[Link](#)

Unbounded: outer coins, inner amounts forward.

Subset Sum

Partition Equal Subset Sum

LEETCODE

[Link](#)

Subset-sum DP toward total/2.

Partition Equal Subset Sum

Partition Equal Subset Sum

LEETCODE

[Link](#)

If total odd impossible; else knapsack reach total/2.

Coin Change

Coin Change

LEETCODE

[Link](#)

Min coins DP: $dp[x] = \min \text{ over coins of } 1 + dp[x - c]$.

Coin Change II

Coin Change II

LEETCODE

[Link](#)

Count combinations (order doesn't matter) with coin-outer loop.

Target Sum

Target Sum

LEETCODE

[Link](#)

Reduce to subset sum: partition into $+/â''$ sets.

5. Subsequence DP

Longest Common Subsequence

Longest Common Subsequence

[LEETCODE](#)[Link](#)

2D DP: match $\hat{a} + 1 + \text{diag}$; else $\max(\text{skip either char})$.

Longest Increasing Subsequence

Longest Increasing Subsequence

[LEETCODE](#)[Link](#)

$O(n^2)$ DP or patience sorting / tails binary search $O(n \log n)$.

Longest Palindromic Subsequence

Longest Palindromic Subsequence

[LEETCODE](#)[Link](#)

$\text{LCS}(s, \text{reverse}(s))$, or interval DP on $i..j$.

Distinct Subsequences

Distinct Subsequences

[LEETCODE](#)[Link](#)

$\text{dp}[i][j]$ ways $s[:i]$ forms $t[:j]$.

Number of Subsequences

Number of Longest Increasing Subsequence

LEETCODE

[Link](#)

Alongside LIS length DP, track count of ways to reach that length.

6. String DP

Edit Distance

Edit Distance

[LEETCODE](#)[Link](#)

dp insert/delete/replace; classic Levenshtein recurrence.

Word Break

Word Break

[LEETCODE](#)[Link](#)

dp[i] true if some word ends at i and dp[start] true.

Palindrome Partitioning

Palindrome Partitioning

[LEETCODE](#)[Link](#)

Backtracking + isPalindrome checks (or DP cuts for min cuts variant).

Wildcard Matching

Wildcard Matching

[LEETCODE](#)[Link](#)

DP for '?' and '*'; '*' can match empty or extend previous.

7. Stock DP

One Transaction

Best Time to Buy and Sell Stock

LEETCODE

[Link](#)

Track min price so far; max profit = $\text{price} - \text{min}$.

Unlimited Transactions

Best Time to Buy and Sell Stock II

LEETCODE

[Link](#)

Sum all upward adjacent differences (or hold/cash DP).

At Most K Transactions

Best Time to Buy and Sell Stock IV

LEETCODE

[Link](#)

DP by transaction count k : hold vs cash states.

Cooldown

Best Time to Buy and Sell Stock with Cooldown

LEETCODE

[Link](#)

States: hold, sold (cooldown), rest — transition carefully.

Transaction Fee

Best Time to Buy and Sell Stock with Transaction Fee

LEETCODE

[Link](#)

Unlimited transactions DP; subtract fee when selling.

8. Interval / Partition DP

Matrix Chain Multiplication

Burst Balloons

[LEETCODE](#)[Link](#)

Interval DP cousin of MCM: decide last burst in (L,R).

Burst Balloons

Burst Balloons

[LEETCODE](#)[Link](#)

Pad 1s; $dp[l][r]$ max coins bursting open interval.

Palindrome Partitioning

Palindrome Partitioning II

[LEETCODE](#)[Link](#)

DP min cuts; precompute palindrome table for substrings.

Partition Problems

Partition Array for Maximum Sum

[LEETCODE](#)[Link](#)

$dp[i]$ = max over last partition length $\leq k$ ending at i .

9. Tree DP

DP on Binary Tree

Binary Tree Maximum Path Sum

LEETCODE

[Link](#)

Tree DP: return gain upward; update global for path through node.

House Robber on Tree

House Robber III

LEETCODE

[Link](#)

Each node returns [rob, skip] pair from children.

Maximum Independent Set

House Robber III

LEETCODE

[Link](#)

Maximum independent set on a tree $\hat{=}$ rob/skip DP.

Rerooting Concept

Sum of Distances in Tree

LEETCODE

[Link](#)

Two DFS: subtree sizes, then reroot formula to all nodes.

10. Bitmask DP

TSP

Find the Shortest Superstring

LEETCODE

[Link](#)

Bitmask TSP-style DP over subsets of strings.

Assignment Problems

Shortest Path Visiting All Nodes

LEETCODE

[Link](#)

Assignment/TSP flavor: BFS on (node, mask).

Subset State

Partition to K Equal Sum Subsets

LEETCODE

[Link](#)

Subset DP / bitmask over filled buckets.

State Compression

Shortest Path Visiting All Nodes

LEETCODE

[Link](#)

State compression = encode visited set in an int mask.

11. Advanced DP

Digit DP

Numbers At Most N Given Digit Set

LEETCODE

[Link](#)

Digit DP by length and tight prefix vs N.

DP on DAG

Longest Increasing Path in a Matrix

LEETCODE

[Link](#)

Memoized DFS on DAG of increasing cells.

DP with Monotonic Queue

Jump Game VI

LEETCODE

[Link](#)

DP + monotonic deque of best previous scores in window k.

DP Optimization

Burst Balloons

LEETCODE

[Link](#)

Interval DP is the gateway to more advanced DP optimizations.

Divide & Conquer DP

Split Array Largest Sum

LEETCODE

[Link](#)

Binary search on answer (related family to D&C DP optimizations).

Knuth Optimization

Minimum Cost to Cut a Stick

LEETCODE

[Link](#)

Interval DP on cuts; Knuth optimization applies to similar recurrences.

Convex Hull Trick

Funny Functions / CHT practice (GFG)

GFG

[Link](#)

When DP transition is min over lines $mx+b$, maintain a convex hull of lines.

Built for ChampDSA interview prep. Hints are approach nudges only – in an interview you must derive the solution. Prefer solving before peeking. Sources: LeetCode, GeeksforGeeks, and classic Striver-style coverage.